

カブナビ（仮）

開発者向け仕様書 v1.0

1. プロジェクト概要

プロジェクト名

カブナビ（仮）

ジャンル

Webアプリケーション

対応ゲーム

あつまれ どうぶつの森

目的

「あつまれ どうぶつの森」のカブ価を記録し、入力された価格から今後の価格を予測して、ユーザーへ最適な売却タイミング・購入判断を提供する。

2. 開発方針

- スマートフォンを最優先としたUI
 - PC・タブレットにも対応
 - ログインなしでも利用可能
 - Googleログインでデータ同期
 - 無料で運用できる構成
 - 保守・拡張しやすい設計
 - レスポンスを高速化
 - オフラインでも基本機能が利用可能
-

3. 技術スタック

フロントエンド

- React
- TypeScript

- Vite
- Tailwind CSS

バックエンド

- Supabase

利用機能

- PostgreSQL
- Authentication
- Row Level Security

グラフ

- Chart.js

状態管理

- Zustand

バリデーション

- Zod

デプロイ

- Vercel

バージョン管理

- GitHub

4. ブランチ運用

main
本番環境

develop
開発環境

feature/*
新機能開発

fix/*
バグ修正

5. コーディングルール

命名

コンポーネント

PascalCase

例

HomePage.tsx

InputCard.tsx

変数

camelCase

例

buyPrice

currentWeek

定数

UPPER_SNAKE_CASE

例

MAX_PRICE

DEFAULT_THEME

CSS

Tailwind CSSを使用する。

CSSファイルは最小限にする。

6. 開発環境

Node.js LTS

Visual Studio Code

Git

npm

7. ディレクトリ構成

src/

components/

pages/

layouts/

hooks/

services/

store/

types/

utils/

assets/

App.tsx

main.tsx

8. 開発ルール

- TypeScriptの型定義を必須とする。
- any型は禁止
- コンポーネントは再利用可能な設計とする。
- 共通処理はutilsへ配置する。
- API処理はservicesへまとめる。
- 状態管理はZustandで行う。

- Firestore等は使用せず、Supabaseへ統一する。
-

9. 品質基準

- ESLintエラー 0件
 - TypeScriptエラー 0件
 - Console Error 0件
 - Console Warning 0件
 - Lighthouse Performance 90点以上
 - モバイル表示を優先
 - レスポンシブ対応
-

10. 今後作成する設計書

- システム仕様書
 - 画面設計書
 - データベース設計書
 - API設計書
 - Supabase設計書
 - 認証設計書
 - UIデザインガイド
 - コンポーネント設計書
 - 予測アルゴリズム設計書
 - テスト仕様書
 - リリース手順書
 - 運用・保守手順書
-

本資料は開発の基準となる文書であり、仕様変更時は必ず更新すること。

カブナビ（仮）

画面設計書 v1.0

1. 画面一覧

画面ID	画面名	概要
SCR-001	ホーム	今週のカブ価・予測・アドバイスを表示
SCR-002	価格入力	購入価格・午前・午後価格を入力
SCR-003	履歴一覧	過去の週を一覧表示
SCR-004	履歴詳細	過去の週の詳細を表示
SCR-005	設定	アプリ設定・ログイン
SCR-006	ヘルプ	使い方・FAQ

SCR-001 ホーム

目的

ユーザーがアプリを開いて最初に表示される画面。

現在のカブ価状況と売買判断を一目で確認できる。

レイアウト

ヘッダー

表示項目

- ・アプリロゴ
 - ・アプリ名
 - ・現在の週
 - ・ログイン状態
-

カード① 今週の情報

表示

- 購入価格
 - 購入株数
 - 今日の価格
 - 最終更新日時
-

カード② 予測

表示

- 現在の予測パターン
 - 各パターンの確率
 - 最高予想価格
 - 最高予想日時
-

カード③ 売り時アドバイス

表示

- 判定
- (売る・待つ・注意)
- 理由
 - 推奨タイミング
-

カード④ 買い時アドバイス

日曜日のみ表示

表示

- 評価 (★)
 - コメント
-

カード⑤ 利益シミュレーション

表示

- 現在売った場合の利益
 - 利益率
 - 売却金額
-

グラフ

折れ線グラフ

表示

- 実際の価格
 - 予測価格
-

フッターメニュー

ホーム

入力

履歴

設定

SCR-002 価格入力

目的

価格入力・編集

入力フォーム

日曜日

購入価格

購入株数

月曜日～土曜日

曜日ごとに

午前価格

午後価格

を入力する。

ボタン

保存

ホームへ戻る

入力制限

数字のみ

0～999

空欄可

入力時は自動保存

SCR-003 履歴一覧

表示内容

一覧形式

- ・年
- ・週
- ・購入価格
- ・最高価格
- ・利益

選択すると詳細画面へ遷移

SCR-004 履歴詳細

表示

- ・購入価格
 - ・購入株数
 - ・価格推移
 - ・グラフ
 - ・最高価格
 - ・利益
 - ・売却価格
 - ・予測結果
-

SCR-005 設定

アカウント

Googleログイン

ログアウト

データ

データ同期

JSONエクスポート

JSONインポート

全データ削除

表示

ダークモード

テーマ

言語

その他

利用規約

プライバシーポリシー

バージョン

ライセンス

SCR-006 ヘルプ

内容

カブについて

使い方

予測について

よくある質問

お問い合わせ

更新履歴

共通仕様

レスポンス

スマートフォン優先

対応サイズ

- 360px
- 390px
- 412px
- 768px
- 1024px
- 1440px

カラー

メインカラー

緑

アクセント

黄色

成功

緑

注意

オレンジ

危険

赤

背景

ライトモード

ダークモード

ボタン

高さ

48px以上

角丸

12px

押下アニメーションあり

カード

角丸

16px

影あり

余白

16px

フォント

本文

16px

見出し

24px

サブタイトル

18px

アイコン

Material Symbolsを使用

画面遷移

ホーム



価格入力



保存



ホーム

ホーム



履歴一覧



履歴詳細



ホーム

ホーム



設定



ホーム

エラー表示

入力エラー

「0～999の数字を入力してください。」

通信エラー

「通信に失敗しました。」

同期エラー

「データ同期に失敗しました。」

今後追加予定

- 通知画面
- ランキング画面
- フレンド共有画面
- AI分析画面
- 統計画面

カブナビ（仮）

API設計書 v1.0

1. 概要

本アプリはSupabaseをバックエンドとして使用する。

フロントエンドからSupabase JavaScript SDKを利用し、認証・データ取得・更新・削除を行う。

REST APIサーバーは実装しない。

2. 認証

方式

Supabase Authentication

対応ログイン

- ・ Googleログイン

将来対応

- ・ 匿名ログイン
 - ・ Appleログイン
-

3. API一覧

API名	内容
login	Googleログイン
logout	ログアウト
getCurrentUser	ログインユーザー取得
getCurrentWeek	今週データ取得
saveWeek	今週データ保存
updatePrice	価格更新

API名	内容
getHistory	履歴取得
deleteWeek	週データ削除
exportData	JSON出力
importData	JSON読込

4. login

目的

Googleログインを実行する。

戻り値

- ユーザー情報
- セッション情報

エラー

- ログイン失敗
- キャンセル

5. logout

目的

ログアウトする。

処理

セッション削除

ホーム画面へ戻る。

6. getCurrentUser

目的

現在ログイン中のユーザーを取得する。

戻り値

- id
- displayName
- email
- avatar

ログインしていない場合

null

7. getCurrentWeek

目的

現在週のデータ取得

入力

年

週番号

戻り値

- 購入価格
- 購入数
- 午前価格
- 午後価格
- 予測
- アドバイス
- 利益

データが存在しない場合

新規作成する。

8. saveWeek

目的

週データ保存

保存内容

購入価格

購入数

価格一覧

予測結果

利益

更新日時

戻り値

保存成功

9. updatePrice

目的

価格更新

入力

曜日

時間帯

価格

処理

価格保存

↓

予測再計算

↓

利益再計算

↓

画面更新

戻り値

更新済みデータ

10. getHistory

目的

履歴取得

並び順

新しい順

取得件数

初期20件

追加読み込み対応

戻り値

週一覧

11. deleteWeek

目的

週データ削除

確認ダイアログ表示後に削除

論理削除ではなく物理削除

12. exportData

目的

JSONとして保存

ファイル名

kabunavi_backup.json

内容

設定

履歴

入力データ

予測

13. importData

目的

JSON読込

処理

形式チェック

↓

重複確認

↓

データ保存

↓

完了メッセージ

14. エラーコード

コード	内容
AUTH001	ログイン失敗
AUTH002	認証期限切れ
DATA001	データ取得失敗
DATA002	保存失敗
DATA003	削除失敗
FILE001	JSON形式エラー
NETWORK001	通信エラー

15. 共通仕様

レスポンスはTypeScript型で管理する。

すべて非同期処理（async/await）で実装する。

例外はtry/catchで処理する。

ユーザーへ分かりやすいエラーメッセージを表示する。

16. キャッシュ

現在週データはメモリキャッシュを利用する。

履歴は必要時のみ取得する。

不要な再読み込みを避ける。

17. セキュリティ

Row Level Security（RLS）を有効化する。

ログインユーザー本人のみ読み書き可能とする。

SQLはSupabase SDK経由で実行し、認証情報を直接扱わない。

環境変数（SUPABASE_URL、SUPABASE_ANON_KEY）はクライアントコードへハードコーディングしない。

18. 将来追加予定

- ・ フレンド共有API
- ・ ランキングAPI
- ・ 通知API
- ・ AI分析API
- ・ 統計API
- ・ イベントAPI

カブナビ（仮）

カブ価予測アルゴリズム設計書 v1.0

1. 概要

本アプリでは、カブ価予測アルゴリズムを一から実装しない。

既存のオープンソース実装を利用または参考にし、アプリへ組み込む。

候補実装

Turnip Calculator <https://github.com/elxris/Turnip-Calculator>

※ 利用前にライセンスを確認し、必要な表記・条件を遵守すること。

2. 目的

入力されたカブ価から、今週の価格変動パターンを推定し、ユーザーへ売却判断を支援する。

3. 入力データ

日曜日

- ・購入価格

月曜日～土曜日

- ・午前価格 ・午後価格

未入力データは null として扱う。

4. 出力データ

予測パターン

- ・大型跳ね型 ・小型跳ね型 ・波型 ・減少型

各パターンの確率

最高予想価格

最高予想曜日

最高予想時間帯

今後の予想価格一覧

5. 処理タイミング

以下のタイミングで予測を再実行する。

・購入価格入力時 ・価格入力時 ・価格変更時 ・データ読込時

処理はリアルタイムで実行する。

6. アプリ側の責務

予測ロジックは既存ライブラリへ委譲する。

アプリ側では以下を担当する。

・入力データの整形 ・予測結果の取得 ・グラフ表示 ・売り時アドバイス表示 ・利益計算 ・エラーハンドリング

7. エラー処理

予測不能な場合は以下を表示する。

「予測に必要なデータが不足しています。」

例

・購入価格未入力 ・入力データが不足 ・不正な価格

8. 売り時判定

予測結果を基にアプリ側で判定する。

表示例

・今売るのがおすすめ ・もう少し待ちましょう ・価格が下がる可能性があります

9. パフォーマンス

予測処理はブラウザ上で完結する。

不要な再計算は行わず、入力に変更された場合のみ再実行する。

10. 保守方針

予測アルゴリズム部分は他の処理と分離する。

例

`src/services/prediction/`

`prediction.ts`

`turnipCalculator.ts`

`adapter.ts`

将来的に別のアルゴリズムへ差し替えられる設計とする。

11. ライセンス

使用するオープンソースのライセンスを確認し、必要な著作権表示やライセンス表記を行うこと。

ライセンス変更があった場合は、影響を確認して対応すること。

カブナビ（仮）

Supabase設計書 v1.0

プロジェクト概要

項目	内容
データベース	Supabase PostgreSQL
認証	Supabase Auth (Google)
ホスティング	Vercel
フロントエンド	React + TypeScript + Vite
状態管理	Zustand
グラフ	Chart.js
CSS	Tailwind CSS

データベース構成

```
users
├── weeks
│   ├── prices
│   └── predictions
```

users

ユーザー情報

カラム	型	必須	説明
id	UUID	✓	Supabase User ID
display_name	TEXT		表示名
created_at	TIMESTAMP	✓	登録日時

weeks

1週間分のデータ

カラム	型	必須	説明
id	UUID	✓	週ID
user_id	UUID	✓	所有ユーザー
year	INTEGER	✓	年
week	INTEGER	✓	週番号
buy_price	INTEGER	✓	購入価格
buy_count	INTEGER		購入株数
created_at	TIMESTAMP	✓	作成日時
updated_at	TIMESTAMP	✓	更新日時

prices

午前・午後の価格

カラム	型	説明
id	UUID	ID
week_id	UUID	weeks参照
day	TEXT	Monday～Saturday
period	TEXT	morning / afternoon
price	INTEGER	価格

predictions

予測結果

カラム	型	説明
id	UUID	ID
week_id	UUID	weeks参照
pattern	TEXT	予測パターン

カラム	型	説明
best_price	INTEGER	最高予想価格
best_day	TEXT	最高予想曜日
best_period	TEXT	午前・午後
large_spike	INTEGER	大型跳ね確率
small_spike	INTEGER	小型跳ね確率
fluctuating	INTEGER	波型確率
decreasing	INTEGER	減少型確率

認証

ログイン

- Googleログイン
- セッションはSupabase Authで管理

将来対応

- 匿名ログイン
- Appleログイン

Row Level Security (RLS)

全テーブルで有効化

操作	条件
SELECT	auth.uid() = user_id
INSERT	auth.uid() = user_id
UPDATE	auth.uid() = user_id
DELETE	auth.uid() = user_id

データ保存フロー

ゲスト利用

|



フォルダ構成

```
src/
├── components/
├── pages/
├── layouts/
├── hooks/
├── services/
│   ├── supabase.ts
│   ├── weekService.ts
│   ├── priceService.ts
│   └── predictionService.ts
├── store/
├── types/
├── utils/
├── App.tsx
└── main.tsx
```

環境変数

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
```

※ `.env` はGitHubへコミットしない。

デプロイ

項目	内容
ホスティング	Vercel
データベース	Supabase
ソース管理	GitHub

GitHubへPushするとVercelが自動デプロイを実行する。

使用ライブラリ

ライブラリ	用途
@supabase/supabase-js	Supabase接続
React Router	画面遷移
Zustand	状態管理
Zod	入力チェック
Chart.js	グラフ
Tailwind CSS	デザイン

将来追加予定

- Realtime同期
- Storage (画像保存)
- Push通知
- Edge Functions
- AIアドバイス
- フレンド共有
- ランキング
- 統計画面

開発方針

- サーバー (Node.js + Express) は構築しない
- Supabaseを唯一のバックエンドとする
- クライアント側で予測アルゴリズムを実行する
- TypeScriptを必須とし、`any`型は使用しない
- ESLint・Prettierを導入し、コード品質を維持する

- モバイルファーストで設計する